

3EC – 1º Global Solution 2026

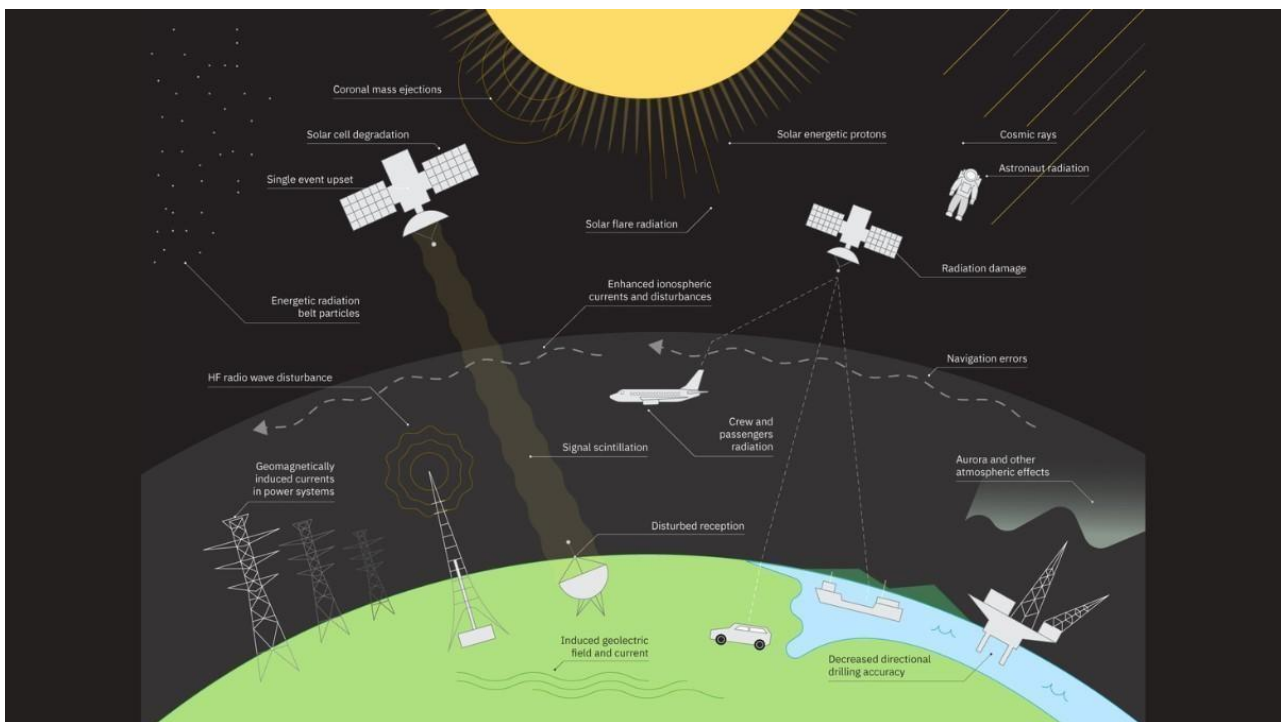
Sistemas Operacionais

Orbital Mission Control System - Performance Analysis

Integrantes:

Nome	RM
Gabriel Lacerda Covello Arimatéa	556391
Mayene Gabrielle Aragão Padilha Doria	558858

A tarefa de monitorar e diagnosticar a saúde de espaçonaves está se tornando crítica. À medida que os satélites e outras espaçonaves se tornaram mais complexos, a quantidade de dados que enviam de volta, incluindo informações sobre o desempenho de praticamente todos os componentes, aumentou exponencialmente. Enquanto isso, cada vez mais operadores lançam não apenas satélites individuais, mas constelações inteiras que podem incluir dezenas de orbitadores. As equipes em solo precisam lidar com esse dilúvio de dados por meio de interfaces baseadas em texto em diversos programas de computador diferentes, simultaneamente.



Para lidar com essa complexidade crescente, sistemas utilizando inteligência artificial para mineração de dados recebidos pelos sistemas de controle de missão orbital têm sido desenvolvidos, para modelar e prever todos os comportamentos possíveis de forma autônoma. Contudo, essa solução depende de cluster de servidores de alta performance e com comportamento determinístico - sistemas nos quais, sob as mesmas condições iniciais, o sistema sempre produzirá a mesma resposta ou resultado.



Assim, considere que na fase de projeto de um sistema de controle de missão orbital seja necessário analisar as situações a seguir.

1) **(2,0 pontos)** Considere um sistema operacional que utilize escalonamento circular com fatia de tempo igual a 10 ut (unidades de tempo), com tempo para troca de contexto desprezível e em execução em um computador com apenas um processador. No momento de tempo inicial existem apenas três processos (P1, P2 e P3) na fila de processos prontos e o tempo de CPU de cada processo é de, respectivamente, 10, 10 e 16 ut. Faça o gráfico de execução destes três processos em função do tempo, considerando:

a) Os processos P1, P2 e P3 são executados nesta ordem.

Resposta: Esta regra define o estado inicial do sistema no instante $t = 0$. A fila de processos prontos é formada estritamente como [P1, P2, P3]. Como P1 está na cabeça da fila, ele será o primeiro processo a receber a CPU para processamento, enquanto P2 e P3 ficam em estado de espera.

b) Que o processo P1 realiza duas operações de E/S: a primeira operação de E/S é executada após 2 ut de processamento e consome 15 ut para ser concluída e a segunda operação de E/S é executada após 5 ut de processamento e consome 5 ut para ser concluída.

Resposta: Esta regra mapeia todo o ciclo de vida do processo P1 (que possui um tempo total de CPU de 10 ut). Baseado no tempo acumulado de processamento, o comportamento dele será:

- Inicia na CPU e executa 2 ut.
- Sofre o primeiro bloqueio por E/S (que demorará 15ut em background).
- Ao retornar a CPU, executa mais 3 ut (atingindo a marca de 5 ut acumulados de processamento).
- Sofre o segundo bloqueio por E/S (que demorará 5 ut).
- Retorna a CPU, executa seus 5 ut restantes, atingindo os 10 ut totais, e é finalizado.

c) Que o processo P2 realiza duas operações de E/S: a primeira operação de E/S é executada após 3 ut de processamento e consome 13 ut para ser concluída e a segunda operação de E/S é executada após 8 ut de processamento e consome 3 ut para ser concluída.

Resposta: Esta regra mapeia todo o ciclo de vida do processo P2 (que também possui um tempo total de CPU de 10 ut). O comportamento dele será:

- Quando herda a CPU, executa 3 ut.
- Sofre o primeiro bloqueio por E/S (que demorará 13 ut).
- Ao retornar para a CPU, executa mais 5 ut (atingindo a marca de 8 ut acumulados de processamento).
- Sofre o segundo bloqueio por E/S (que demorará 3 ut).
- Ao retornar para a CPU, executa seus 2 ut restantes, atingindo os 10 ut totais, e é finalizado.

d) Que o processo P3 realiza uma operação de E/S, executada após 14 ut de processamento e consome 4 ut para ser concluída.

Resposta: Esta regra dita o comportamento do P3 (tempo total de 16 ut) e nos permite montar o Gráfico de Execução final com todas as interações. O comportamento do P3 será:

- Executa 10 ut diretos (esgotando sua fatia de tempo/Quantum). Ele sofre preempção, mas como a fila de prontos estará vazia neste instante específico (P1 e P2 estão em E/S), ele continua na CPU.
- Executa mais 4 ut (atingindo 14 ut acumulados).
- Sofre seu único bloqueio por E/S (que demorará 4 ut).
- Ao retornar, executa seus 2 ut finais e é concluído.

GRAFICO DE EXECUÇÃO DO PROCESSAMENTO EM RELAÇÃO AO TEMPO

Temp(ut)	0 a 2	2 a 5	5 a 19	19 a 22	22 a 27	27 a 29	29 a 34	34 a 36
Processo na CPU	P1	P2	P3	P1	P2	P3	P1	P2

2) **(4,0 pontos)** Em um *benchmarking* – processo de comparação de produtos, serviços e práticas corporativas – 6 aplicações de data-mining foram avaliadas: App1, App2, App3, App4, App5 e App6. O ambiente de teste é um servidor equipado com um processador de um núcleo (*single core*) que serializa os acessos ao disco rígido. Além do sistema operacional multitarefa e dos processos do sistema, este servidor executa apenas as aplicações que você solicitar.

Durante a avaliação, você percebeu que algumas aplicações passam, praticamente, 100% do tempo de execução realizando operações lógicas e aritméticas (denominadas aplicações **CPU-bound**) e outras aplicações passam cerca de 70% do tempo de execução realizando acesso ao disco rígido (denominadas aplicações **I/O-bound**) e 30% do tempo de execução realizando operações lógicas e aritméticas.

Como parte da avaliação, inicialmente, você executou as aplicações isoladamente e mediu o tempo de execução de cada uma delas (Tabela I). Em seguida, as aplicações foram combinadas em grupos e executadas simultaneamente (Tabela II).

Tabela I

aplicação	tipo	tempo de execução [ms]
App1	<i>CPU-bound</i>	20
App2	<i>CPU-bound</i>	40
App3	<i>CPU-bound</i>	60
App4	<i>I/O-bound</i>	50
App5	<i>I/O-bound</i>	100
App6	<i>I/O-bound</i>	150

Tabela II

GRUPO	tempo de execução [ms]
1: App1 + App2 + App3	125
2: App4 + App5 + App6	210
3: App3 + App5 + App6	160
4: TODAS	320

Alguns dias depois, quando você foi elaborar o relatório final da análise, revendo as tabelas, você ficou com dúvida em relação aos tempos anotados. Com base nos dados das Tabela I e II, avalie as afirmações a seguir, indicando se cada afirmação é VERDADEIRA ou FALSA, **justificando vossas respostas** em cada uma delas.

- APP1 (CPU): CPU = 20 ms | Disco = 0 ms
- APP2 (CPU): CPU = 40 ms | Disco = 0 ms
- APP3 (CPU): CPU = 60 ms | Disco = 0 ms
- APP4 (I/O): CPU = 15 ms (30%) | Disco = 35 ms (70%)
- APP5 (I/O): CPU = 30 ms (30%) | Disco = 70 ms (70%)
- APP6 (I/O): CPU = 45 ms (30%) | Disco = 105 ms (70%)

(A) É possível que o tempo total do GRUPO 4 seja 320 ms.

Resposta: Verdadeira.

Justificativa: O Grupo 4 executa todas as 6 aplicações simultaneamente. Para verificar se 320 ms é um tempo fisicamente possível, precisamos calcular os limites inferior (mínimo teórico) e superior (máximo teórico) de execução concorrente do sistema.

- **Limite Mínimo Teórico**
 - Demanda total de **CPU** do grupo: $20 + 40 + 60 + 15 + 30 + 45 = 210$ ms.
 - Demanda total de **Disco** do grupo: $0 + 0 + 0 + 35 + 70 + 105 = 210$ ms.
- **Limite Máximo Teórico (Pior cenário)**
 - Soma total: $20 + 40 + 60 + 50 + 100 + 150 = 420$ ms.

Como o valor de 320 ms está contido dentro do intervalo fisicamente realizável pelo hardware ($210 \text{ ms} < 320 \text{ ms} < 420 \text{ ms}$), a afirmação é verdadeira.

(B) O tempo total do GRUPO 1 deveria ser, no mínimo, de 80 ms.

Resposta: Falsa.

Justificativa: O Grupo 1 é composto apenas pelas aplicações App1, App2 e App3, que são 100% CPU-bound. Como o ambiente de teste possui apenas um processador de um único núcleo (single-core), o processador não consegue executar duas instruções computacionais de forma paralela. Por isso, o tempo não seria 80 ms.

(C) O tempo total do GRUPO 2 deveria ser, no mínimo, de 250 ms.

Resposta: Falsa.

Justificativa: O Grupo 2 é composto pelas aplicações App4, App5 e App6, todas I/O-bound. Para determinar o tempo mínimo de execução delas em grupo, analisamos o limite físico dos recursos de CPU e Disco necessários para elas:

- **Total de tempo de CPU:** $15 \text{ ms} + 30 \text{ ms} + 45 \text{ ms} = 90$ ms.
- **Total de tempo de Disco:** $35 \text{ ms} + 70 \text{ ms} + 105 \text{ ms} = 210$ ms.

Como o acesso ao disco rígido é **serializado** (um processo por vez na fila do disco), o hardware de armazenamento precisará funcionar por, no mínimo, 210 ms acumulados para atender às três aplicações. Por tanto, os 90 ms em CPU podem ser processados e ocultados em paralelo enquanto o disco rígido trabalha para as outras aplicações.

(D) O tempo total do GRUPO 3 deveria estar acima de 200 ms e abaixo de 300 ms.

Resposta: Verdadeira.

Justificativa: O grupo 3 é composto por APP3 (CPU), APP5 (I/O) e APP6 (I/O), vamos calcular os requisitos mínimos e máximos isolados para este grupo:

- **Total de CPU exigido:** 60 ms (app3) + 30 ms (app5) + 45 ms (app6) = 135 ms.
- **Total de Disco exigido:** 0 ms (app3) + 70ms (app5) + 105 ms (app6) = 175 ms.

Mesmo sob uma sobreposição perfeita (cpu trabalhando em paralelo com o disco), o disco rígido precisa de 175 ms de trabalho serializado de forma obrigatória. Além disso, o App3 exige 60 ms de CPU pura. Por tanto, o mínimo exigido seria algo acima de 200 ms. Entretanto, em execução sequencial, o tempo seria: 60 + 100 + 150 = 310 ms, logo, em modo multitarefa o tempo final seria inferior a 310 ms.

3) **(1,5 ponto)** Medidas efetuadas em um determinado sistema operacional mostraram que, em média, um processo é executado por um tempo T antes de ser bloqueado por operações de entrada e saída (E/S). A troca de contexto é feita em um tempo S que se constitui, obviamente, em uma sobrecarga (*overhead*). Para um escalonamento circular com fatia de tempo Q (*quantum*), associe o valor ou a fórmula que representa a eficiência do processador deste sistema para cada um dos valores de Q listados abaixo.

Contexto: A eficiência do processador é dada pela razão entre o tempo em que ele está realmente processando a aplicação (Tempo Útil) e o tempo total gasto no ciclo (Tempo Útil + Tempo de Troca de Contexto).

A fórmula base é:

$$ef = \frac{T_u}{T_u + S}$$

(A) $S < T < Q$

Resposta: O tempo que o processo leva para solicitar uma Entrada/Saída (T) é menor que o quantum (Q). Isso significa que o processo nunca sofrerá com preempção pelo esgotamento do quantum, ele sempre irá parar voluntariamente para fazer E/S após executar por um tempo T. O tempo útil (T_u) será igual a T. A cada ciclo, o processador gasta o tempo T executando o processo e o tempo S trocando de contexto.

Fórmula: ef = eficiência do processador

$$ef = \frac{T}{T + S}$$

(B) $S < Q < T$

Resposta: O quantum (Q) agora é menor que o tempo médio para E/S (T). Portanto, o processo vai esgotar todo o seu *quantum* na CPU e sofrerá preempção (interrupção forçada) antes de conseguir solicitar a E/S.

O tempo útil (Tu) em cada passagem pela CPU será exatamente igual ao *quantum* (Q). O tempo total gasto no ciclo será o tempo do *quantum* mais a troca de contexto (Q + S).

Formula:

$$ef = \frac{Q}{Q + S}$$

(C) $S = Q < T$

Resposta: O quantum é menor que o tempo de E/S, então o tempo útil continua sendo Q. Porém, o tempo gasto para trocar de contexto (S) é exatamente igual ao tempo de processamento (Q).

Formula:

$$ef = \frac{Q}{Q + Q} = \frac{Q}{2Q}$$

O que corresponde a $ef = 0.5$ ou 50%

(D) $Q \rightarrow \infty$

Resposta: Se o quantum tende ao infinito, o escalonamento circular se comporta, na prática, como um sistema não-preemptivo. O processo nunca será interrompido por tempo esgotado. Ele vai rodar até ser bloqueado por E/S, o que ocorre no tempo T.

Formula:

$$ef = \frac{T}{T + S}$$

(E) $Q \rightarrow 0$

Resposta: Se o quantum tende a zero, o processo recebe um tempo ínfimo de CPU antes de sofrer preempção. O tempo útil se aproxima de zero ($T_u \rightarrow 0$), mas a sobrecarga de troca de contexto (S) é um valor físico constante.

Formula:

$$ef = \frac{0}{0 + S}$$

O processador passará praticamente 100% do seu tempo trocando de contexto e 0% processando de fato.

4) **(1,5 ponto)** Considere um cenário em que um computador seja organizado com múltiplos processadores, os quais compartilham a mesma memória RAM. Cada processador possui múltiplos núcleos. Nesse arranjo, o sistema operacional permite múltiplas *threads*, as quais podem ser dinamicamente alocadas para execução em diferentes núcleos e processadores. A partir das informações apresentadas nessa situação, avalie as informações a seguir e assinale (V) verdadeiro ou (F) falso, **justificando vossas respostas** em cada uma delas.

(F) Sistemas com múltiplos processadores devem alocar a mesma quantidade de memória RAM para cada processador do arranjo.

Justificativa: Em uma arquitetura de memória compartilhada a memória RAM funciona como um recurso global e unificado. Não existe a obrigatoriedade de fatiar ou "alocar" quotas fixas de RAM para o processador A ou B. Qualquer processador ou núcleo pode acessar qualquer endereço de memória disponível no sistema dinamicamente, conforme a demanda dos processos que estão executando naquele instante.

(F) Como há múltiplos processadores, são desnecessários os semáforos, uma vez que não há acessos concorrentes a recursos compartilhados.

Justificativa: É exatamente o oposto. A presença de múltiplos processadores gera paralelismo real (instruções executadas no exato mesmo milissegundo). Como eles compartilham a mesma memória RAM, o acesso concorrente e as condições de corrida tornam-se um risco altíssimo. Portanto, semáforos, mutexes e monitores são absolutamente necessários e vitais para sincronizar esses acessos e garantir a integridade dos dados.

(F) Como a exclusão mútua não é possível em arquitetura de múltiplos processadores, apenas uma aplicação pode ser executada de cada vez, mas com múltiplos *threads*.

Justificativa: A exclusão mútua é perfeitamente possível. Ela é garantida através de instruções atômicas de hardware (como Test-and-Set ou Compare-and-Swap) que travam o barramento de memória por ciclos minúsculos de relógio para evitar sobreposição.

(F) Os processos que possuem múltiplos *threads* em execução são mantidos por meio de funções da biblioteca no código da aplicação e dispensam serviços do sistema operacional.

Justificativa: Para que as threads sejam "dinamicamente alocadas para execução em diferentes núcleos e processadores" (como afirma o enunciado), elas obrigatoriamente precisam ser Threads de Nível de Núcleo (Kernel-Level Threads). Se fossem implementadas apenas por bibliotecas em espaço de usuário (User-Level Threads), o Sistema Operacional não saberia da existência delas (enxergaria apenas o processo inteiro) e seria impossível para o SO dividi-las entre vários processadores físicos. Logo, os serviços de escalonamento do Sistema Operacional são indispensáveis.

(V) Dados trocados durante a comunicação entre processos podem ser armazenados nas áreas de memória compartilhada, mas o acesso a essas áreas é intermediado pelo sistema operacional.

Justificativa: A memória compartilhada (*Shared Memory*) é um dos métodos mais eficientes de IPC (Comunicação Entre Processos). O Sistema Operacional atua como o intermediador fundamental: é ele quem cria o segmento de memória, configura as permissões de segurança e mapeia esse segmento físico dentro do espaço de endereçamento virtual de cada processo envolvido.

5) (1,0 ponto) O problema do buffer limitado de tamanho N é um problema clássico de sincronização de processos: um grupo de processos utiliza um buffer de tamanho N para armazenar temporariamente itens produzidos; processos produtores produzem os itens, um a um, e os armazena no buffer; processos consumidores retiram os itens do buffer, um a um, para processamento. O problema do buffer limitado de tamanho N pode ser resolvido com a utilização de semáforos, que são mecanismos de software para controle de concorrência entre processos. Duas operações são definidas para um semáforo (s): wait(s) e signal(s). Considere o problema do buffer limitado de tamanho N cujos pseudocódigos dos processos produtor e consumidor estão mostrados na tabela abaixo. Pode-se resolver esse problema com a utilização dos semáforos mutex, cheio e vazio, inicializados, respectivamente, com 1, 0 e N.

processo produtor	processo consumidor
produz item	comando_e comando_f
comando_a comando_b	retira do buffer
coloca no buffer	comando_g comando_h
comando_c comando_d	consome o item

A partir dessas informações, para que o problema do buffer limitado de tamanho N cujos pseudocódigos foram apresentados acima possa ser resolvido a partir do uso dos semáforos mutex, cheio e vazio, é necessário que comando_a, comando_b, comando_c, comando_d, comando_e, comando_f, comando_g e comando_h correspondam, respectivamente, às operações:

Resposta: Letra A

- (A) **wait(vazio), wait(mutex), signal(mutex), signal(cheio), wait(cheio), wait(mutex), signal(mutex) e signal(vazio)**
- (B) wait(cheio), wait(mutex), signal(mutex), signal(vazio), wait(vazio), signal(mutex), signal(mutex) e wait(cheio)
- (C) wait(mutex), wait(vazio), signal(cheio), signal(mutex), wait(mutex), wait(vazio), signal(cheio) e signal(mutex)
- (D) wait(mutex), wait(vazio), signal(cheio), signal(mutex), wait(mutex), wait(cheio), signal(vazio) e signal(mutex)
- (E) wait(vazio), signal(mutex), signal(cheio), wait(mutex), wait(cheio), signal(mutex), signal(vazio) e signal(mutex)